FLOAT VS FLEXBOX

Quando usare float, inline-block, table o flex?

8-10 minuti — Il Blog di Karen Menezes — 13 aprile 2014

Evito di usare framework CSS nei miei progetti personali, perché li trovo spesso troppo rigidi per il design personalizzato o per il codice finale di produzione. Naturalmente potresti non essere d'accordo.

Detto questo, è sempre una buona idea osservare come i diversi framework gestiscono le griglie. Spesso si vedono selettori di attributo come selector[class*="col-"], che utilizzano un match per sottostringa e possono interferire con altre classi, con !important e con molti valori decimali. Per progetti più piccoli, le griglie sono spesso semplici quanto affiancare (float) una colonna al 75% e una al 25% aggiungendo un po' di padding.

Inoltre, la maggior parte delle griglie pronte all'uso ha breakpoint personalizzati (spesso chiamati mobile/tablet/desktop o small/medium/large), il che non è ideale, perché i breakpoint dovrebbero dipendere da dove il contenuto "si rompe" e non da un'idea arbitraria di dispositivo. Non che non si possano modificare usando mixin tramite preprocessori (su cui molti framework si basano), ma progetti diversi hanno esigenze diverse. Il web è sempre stato fluido, quindi è meglio pensare prima al contenuto e all'usabilità, e poi progettare attorno a questi.

Usare universalmente box-sizing: border-box rende le griglie molto più semplici. Sono un grande fan dell'uso del padding invece dei margini quando possibile, perché border-box include il padding nella larghezza/altezza totale dell'elemento.

Adoro anche calc. Ha un ottimo supporto tra i browser. Con calc, potresti avere 7 box di uguale larghezza dichiarando: width: calc(100% / 7). Fantastico, no?

La maggior parte delle griglie e dei framework front-end che ho visto insiste sull'uso dei float per tutti i layout. Ma a volte non vuoi usare i float. In alcuni casi è meglio usare inline-block, più raramente display: table, e — se il supporto dei browser lo consente — flexbox. Infatti, ho iniziato a usare flexbox in progetti reali per miglioramenti di layout come la centratura verticale (quando è accettabile che i browser più vecchi allineino il contenuto in alto).

Vediamo alcuni vantaggi e svantaggi dei metodi più comuni per layout responsive.

1. Floats

Vantaggi

- Metodo più diffuso per costruire layout; la maggior parte dei framework di griglia lo utilizza.
- I bug dei float sono ben conosciuti e documentati, con soluzioni consolidate.

Svantaggi

- Devono essere "puliti" (cleared). Può diventare complicato se cambi le larghezze su 2-3 breakpoint, perché dovrai ripetere il clear ogni volta. Nessuna centratura verticale.
- Nessuna altezza uniforme.
- Nessuna indipendenza dall'ordine nel markup.

Usalo per:

Grandi blocchi di layout che non richiedono altezze uguali o centratura verticale.

2. Inline Block

Vantaggi

- La centratura verticale è molto utile in alcuni layout.
- Non richiede clear: utile per layout complessi con molti breakpoint.

Svantaggi

- Soffre di spazi bianchi intrinseci, che possono rovinare i calcoli della griglia. Ad esempio, non puoi creare facilmente una griglia a due colonne 30% / 70% con display: inline-block, perché lo spazio bianco può spingere la seconda colonna a capo. Esistono soluzioni: la più semplice è usare liste con doctype HTML5, perché non serve chiudere i tag <li>, eliminando così il problema degli spazi.
- Nessuna altezza uniforme.
- Nessuna indipendenza dall'ordine.

Usalo per:

- Quando ti serve centratura verticale ma non altezze uguali.
- Quando vuoi evitare di fare clear dei float su più breakpoint.
- Quando riesci a gestire il problema degli spazi bianchi senza complicare troppo il lavoro del team.
- Quando il tuo sistema di template non inserisce automaticamente tag <li> di chiusura.
- Quando usi liste per il layout: qui inline-block dà il meglio.

3. Display Table

A volte vuoi altezza uniforme + centratura verticale senza usare JavaScript. Se non puoi usare flexbox, puoi usare display: table. Tuttavia, può essere imprevedibile.

Vantaggi

- Offre centratura verticale e altezze uguali.

Svantaggi

- Richiede div wrapper extra (anche se la specifica dice che non sono necessari, su iOS possono verificarsi bug senza di essi).
- Se vuoi spazio tra celle, devi usare border-collapse: separate sul contenitore, ma questo aggiunge padding ai bordi esterni e può rompere la griglia.
- Il contenuto può fuoriuscire dalle celle ed è difficile da gestire.
- Passare da 4 celle a 2 a 1 in un layout responsive non è semplice (da 4 a 1 sì, cambiando display in block).

Usalo per:

Situazioni delicate in cui servono centratura verticale + altezze uguali con un solo breakpoint (da table a block).

4. Flexbox

Flexbox è fantastico, oltre ogni più sfrenata fantasia di layout CSS.

Vantaggi

- Indipendenza dall'ordine nel markup (molto utile nei layout responsive).
- Elimina la necessità di JavaScript per il layout.
- Centratura verticale.
- Altezze uguali.
- Gestione flessibile degli assi X e Y.
- Gli elementi possono crescere o ridursi, essere righe o colonne, adattarsi allo spazio disponibile.
- Molti modi diversi per ottenere lo stesso risultato.

Svantaggi

- Sintassi inizialmente poco intuitiva.
- Richiede tempo per essere padroneggiato, soprattutto in layout complessi.
- Non funziona nei browser molto vecchi.

Usalo per:

- Centratura verticale.
- Layout indipendenti dall'ordine e con altezze uguali.
- Layout di applicazioni dove gli elementi devono adattarsi dinamicamente.

Browser moderni

Per i browser moderni, l'approccio migliore è:

- **Floats**: quando immagini o media devono "fluttuare" nel testo (come previsto originariamente).
- **inline-block**: per bottoni, input o elementi che richiedono padding/margine ma non devono occupare tutta la larghezza.
- **flex**: per layout di contenuti e componenti (meno ideale per layout di pagina completi, perché troppo "flessibile" per strutture rigide).
- **table**: solo per tabelle. Nient'altro. Mai.